



SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

BİLGİSAYAR MİMARİSİ VE ORGANİZASYONU

3.PROJE ÖDEVİ RAPORU

DERSİN ÖĞRETİM ÜYESİ

Prof. Dr. Halit ÖZTEKİN

HAZIRLAYANLAR

Merve SAY

Gizem KÖSEM

Şevval KARAARSLAN

Hazal KÜÇÜKYEĞEN

Ebrar ÖZDEMİR

KRALİÇE ARI

Bu projede tasarlanan mikrobilgisayar, temel olarak bir **sayı tahmin oyunu** gerçekleştirmektedir. Sistemin amacı, kullanıcının, mikrobilgisayar tarafından rastgele seçilen bir sayıyı tahmin etmesini sağlamaktır. Mikrobilgisayar, başlangıçta belirli bir aralıkta (0–7) rastgele bir sayı üretir. Kullanıcı ise bu sayıyı tahmin etmeye çalışır.

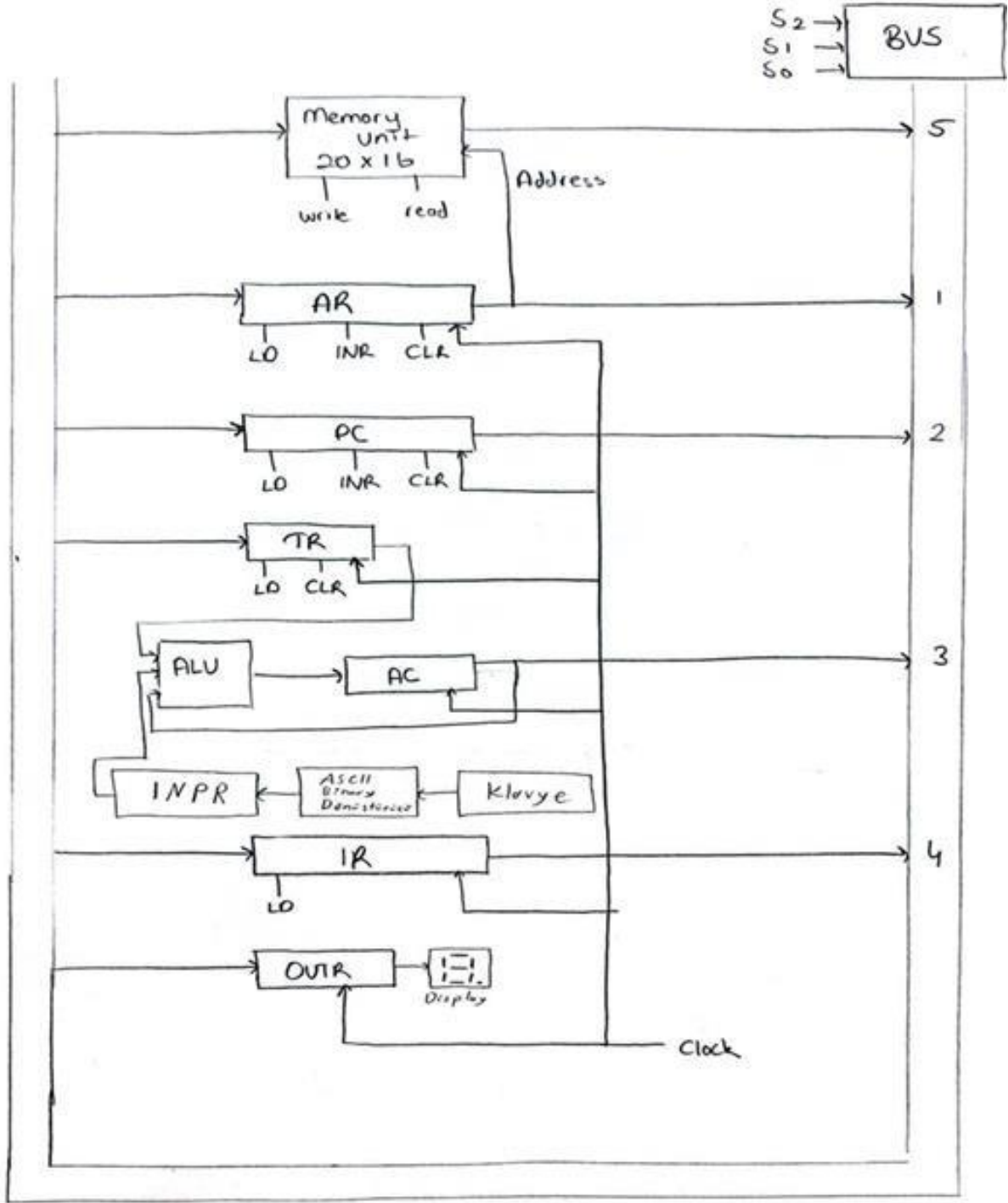
Her tahmin sonrası mikrobilgisayar, kullanıcının girdiği değeri hedef sayı ile karşılaştırarak aşağıdaki çıktılarından birini üretir:

- “k” (küçük): Eğer kullanıcı tahmini, rastgele sayıdan küçükse.
- “b” (büyük): Eğer kullanıcı tahmini, rastgele sayıdan büyükse.
- “d” (doğru): Eğer kullanıcı doğru sayıyı tahmin ettiyse.

Bu çıktı, kullanıcının bir sonraki tahminini daha bilinçli yapmasına yardımcı olur. Mikrobilgisayar bu işlemleri gerçekleştirirken, komut belleği, kontrol birimi, aritmetik-mantık birimi (ALU) ve veri yolu gibi temel bileşenleri kullanır. Sayının kontrolü ve karşılaştırma işlemleri bu birimler arasında veri aktarımı ile gerçekleştirilir.

Sistemimiz genel yapısını ifade eden blok diyagramı ve sistemin çalışmasını daha detaylı şekilde adım adım açıklayan Register Transfer Language (RTL) diyagramı aşağıda sunulmuştur. Blok diyagramı, sistemin temel bileşenlerini ve bu bileşenler arasındaki veri iletim yollarını görselleştirirken; RTL diyagramı ise talimatların nasıl işlendiğini, hangi adımlarda hangi registerların kullanıldığını ve veri akışının nasıl gerçekleştiğini göstermektedir.

Blok Diyagramı



Register Transfer Language (RTL), sistemin clock (clk) sinyali ile senkronize olarak hangi

işlemleri gerçekleştirdiğini, hangi registerların hangi sırayla veri taşıdığını ve ALU ile diğer bileşenlerin nasıl etkileşime girdiğini ayrıntılı bir şekilde göstermektedir. Tasarımda kullanılan temel registerlar ve işlevleri şu şekildedir:

PC (Program Counter): Her saat döngüsünde yürütülecek komutun bellek adresini tutar.

IR (Instruction Register): Bellekten okunan komutu geçici olarak depolar ve kontrol birimine iletir.

AC (Accumulator): ALU işlemlerinin geçici sonuçlarını tutar ve veri işleme sürecinde merkezi bir rol oynar.

AR (Address Register): Bellek erişimlerinde kullanılan adres bilgisini geçici olarak saklar. Genellikle bellekten veri okunmadan veya belleğe veri yazılmadan önce ilgili adres bu register'a yüklenir.

TR (Temporary Register): İşlem sırasında geçici veri saklamak amacıyla kullanılır. Özellikle ALU işlemleri veya veri transferlerinde ara aşamalarda kullanılır.

INPR: Kullanıcının giriş olarak verdiği tahmini saklar.

OUTR: ALU ve karşılaştırma sonuçlarına göre "k", "b" veya "d" karakterlerinden birini ekrana verir.

Bu bileşenler, kontrol biriminden gelen sinyaller doğrultusunda her clock darbesinde belirli işlemleri gerçekleştirerek birbirleriyle veri alışverişinde bulunur. RTL diyagramı, bu veri akışını zaman sıralı olarak göstererek sistemin donanım seviyesindeki işleyişini daha net anlamamızı sağlar.

Aşağıda sunulan RTL diyagramı, sayı tahmin oyununun her bir aşamasında hangi işlem adımlarının gerçekleştiğini, bu adımlarda hangi registerların etkin olduğunu ve kontrol sinyallerinin nasıl yönlendirildiğini şematik olarak göstermektedir.

RTL Diyagramı

FETCH R'T0:AR<-PC
DECODE R'T1:IR<-M[AR], PC<-PC+1
R'T2:D0-D6<-Decode IR (11-13)
AR<-IR (0-10)

KESME

1.GRUP

AND D0T4: TR<-M[AR]
D0T5: AC<-AC $\wedge$ TR, SC<-0
OR D1T4: TR<-M[AR]
D1T5: AC<-AC $\vee$ TR, SC<-0
STA D2T4: M[AR]<-AC, SC<-0
BUN D3T4: PC<-AR, SC<-0
PCP D4T4: PC<-PC+1
ADD D5T4: TR<-M[AR]
D5T5: AC<-AC+TR, SC<-0
LDA D6T4: TR<-M[AR]
D6T5: AC<-TR, SC<-0

2.GRUP D7_T3=R, IR(i)=Bi(i=3,4,5,6,7,8,9,10)

CMA rB10: AC<-AC'
STP rB9: if (AC=0) then OUTR<- 'd'
If (AC>0) then OUTR<- 'b'
if (AC<0) then OUTR<- 'k'
HLT rB8: S<-0
INP rB7: AC (0-7) <-INPR
OUT rB6: OUTR<-AC (0-7)
ION rB5: IEN<-1
IOF rB4: IEN<-0
RND rB3: TR (1,3) <-TR (0,2)
AC (0) <-TR (3) $\vee$ TR (1)
AC(3)<- '0'

MEMORY

HEXADECIMAL KARŞILIĞI

0111000000001110 —01 LDA 14	700E
1111100000001000 -- RND	F004
0001000000001101 —00 STA 13	100D

1111100010000000 -- INP	FC00
1111110000000000 -- CMA	FD00
1010100000000001 --10 ADD 1	A001
0010100000001101 --00 ADD 13	000D
1111101000000000 -- STP	FA00
0010000000000000 -- PCP	0000
0001100000000101 --00 BUN 5	0005
1111100100000000 -- HLT	FB00
0000000000000101 --Random sayının kayıtlı olduğu hücre	
0000000000001101 --Random sayıyı tutan hücrenin adresi	

Kendi Eklediğimiz Komutlar

RND (random) KOMUTU:

RND komutu, temel amacı rastgele sayı üretmek olan özel bir işlemdir. Bu komutun uygulanması sırasında belirli mantıksal ve aritmetik işlemler dizisi gerçekleştirilir. Rastgele sayının oluşturulmasında temel olarak TR ve AC registerları etkin biçimde kullanılır.

İlk aşamada, TR registerının 0-2. bitleri ile 1-3. bitleri arasında bir sola kaydırma (left-shift) işlemi gerçekleştirilir. Bu işlem sonucunda TR'nin içeriği güncellenir ve yeni verinin TR registerına yazılması için Wr_data_TR_internal adlı sinyal devreye girer. Bu sinyal, TR registerına aktarılabacak yeni veriyi taşır.

Sonraki adımda, rastgeleliğin sağlanması için EXOR işlemi uygulanır. Bu işlemde, TR registerının 3. biti ile 1. biti EXOR'lanarak elde edilen sonuç, geçici bir sinyal olan temp değişkenine aktarılır. Elde edilen bu değer, AC registerının 0. bitine yazılır. Ardından, AC registerının 3. biti sıfırlanır. Bu adım, üretilen sayının negatif olmasını önlemek amacıyla gerçekleştirilir; zira işaret bitinin sıfır olması, sayının pozitif olduğunu garanti eder.

Tüm bu işlemlerin sonucunda, rastgele sayı başarılı şekilde üretilmiş olur.

STP (stop) KOMUTU:

Bu komut, tahmin sürecinin tamamlandığını ve yapılan tahminin, daha önceden üretilen rastgele sayıya göre konumunu belirlemeyi amaçlar. Komutun değeri 1 olduğunda, karar süreci AC (Accumulator) registerının içeriğine göre yürütülür:

1. Eğer $AC = 0$ ise bu durum tahminin doğru olduğunu gösterir ve 'd' karakteri, out registerı aracılığıyla ekrana gönderilir.
2. Eğer $AC < 0$ ise bu durum tahminin rastgele sayıdan büyük olduğunu belirtir ve 'b' karakteri ekrana aktarılır.
3. Eğer $AC > 0$ ise tahmin rastgele sayıdan küçük kabul edilir ve 'k' karakteri görüntülenir.

Bu şekilde, sistem kullanıcıya tahmininin doğruluğu hakkında geri bildirim sağlar.

PCP (program counter plus) KOMUTU:

Eğer belirli bir komutun yürütülmesi istenmiyorsa, program sayacı (PC) bir artırılarak ilgili komutun atlanması sağlanır. Bu işlem, komutun yürütülmesini engelleyerek kontrol akışının bir sonraki komuta yönlendirilmesini mümkün kılar.

KARŞILAŞTIRMA MANTIĞI:

Bellekte tutulan rastgele sayı ile kullanıcının girdiği tahmini karşılaştırmak amacıyla çıkarma işlemi gerçekleştirilir. Bu işlemde, rastgele sayıdan kullanıcı tahmini çıkarılır. Çıkarma işlemi, AC registerının 2'si tümleyeni alınarak gerçekleştirilir ve sonuç, kullanıcı tahmini üzerinden değerlendirilir.

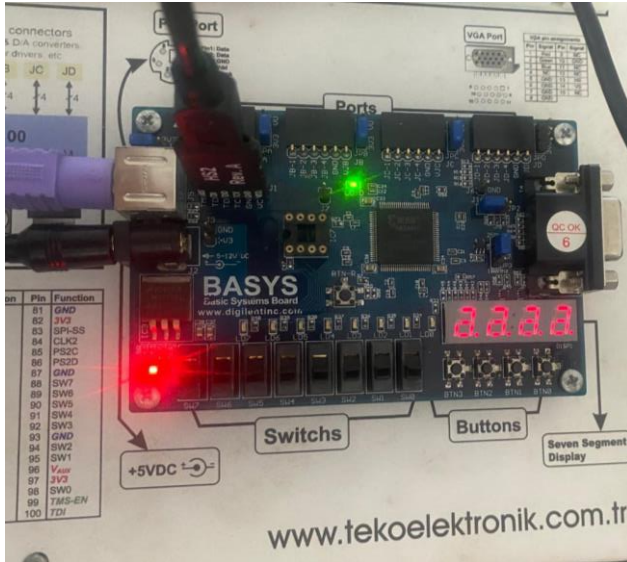
Elde edilen sonuca göre:

- Sonuç **0** ise tahmin doğru kabul edilir ve ekrana '**d**' karakteri görüntülenir.
- Sonuç **negatif** ise kullanıcı tahmini rastgele sayıdan büyük olduğu için '**b**' karakteri görüntülenir.
- Sonuç **pozitif** ise kullanıcı tahmini rastgele sayıdan küçük olduğu için '**k**' karakteri görüntülenir.

FPGA Led Yakma

Bu aşamada, "ekrana veri yazdırma işleminin nasıl gerçekleştiği", "pinlerin işleyiş mantığı" ve "tasarımın FPGA kartına nasıl aktarıldığı" gibi uygulamaya dönük sorulara yanıt bulmak amacıyla kart üzerinde çeşitli testler gerçekleştirilmiştir. Aşağıda, bu testler sonucunda elde edilen örnek çıktılar sunulmuştur:





Proje Aksilikleri ve Alternatifimiz

Projemizin mantıksal yapısını önceki bölümlerde açıkladıktan sonra, ilgili kodları da uygulamaya döktük. Bu kodlar **KraliçeAri** adlı klasörde yer almakta olup, ek dosyalar arasında sunulmuştur. Ancak, birden fazla dosyada aynı bileşenin (component) tekrar çağırılması sonucu **rekürsif (özyinelemeli) tanımlama hataları** ile karşılaştık.

Örneğin, aşağıda gösterilen hata bu duruma ilişkin tipik bir örnektir:

```
/home/ise/KraliceAri/KraliceAri/AR.vhd" line 35: Component 'SAYAC' has been instantiated recursively 64 times. Use "set -recursion_iteration_limit XX" to iterate more.
```

Bu problemi çözmek amacıyla üç farklı yöntem üzerine çalışmalar gerçekleştirdik:

1. Ana (main) modül dışındaki dosyalarda yer alan component tanımlarını kaldırarak, gerekli sinyal ve verileri doğrudan entity üzerinden temin etmeye çalışmak,
2. Tüm temel işlemleri main dosyası içerisinde birleştirerek, diğer modüllerin component çağırısına ihtiyaç duymadan daha yalın çalışmasını sağlamak,
3. Tüm bağlantıları merkezi bir dosyada yaparak modüller arası etkileşimi kontrol altına almak.

Ancak bu yaklaşımların hiçbiri istenilen düzeyde başarılı olamamış ve hatanın tamamen giderilmesini sağlayamamıştır.

Bunun üzerine, FPGA kartı üzerindeki temel kontrol bilginizi ve uygulama yetkinliğimizi göstermek amacıyla alternatif olarak basit bir uygulama geliştirdik. Bu uygulama, kart üzerinde rastgele üretilen bir sayının ekrana yazdırılmasını sağlamaktadır. Bu alternatif yöntemin kodunu da ek dosyada görebilirsiniz. Aşağıda da simülasyon görseli bulunmaktadır:

